

Simulación de un Cardumen de Peces

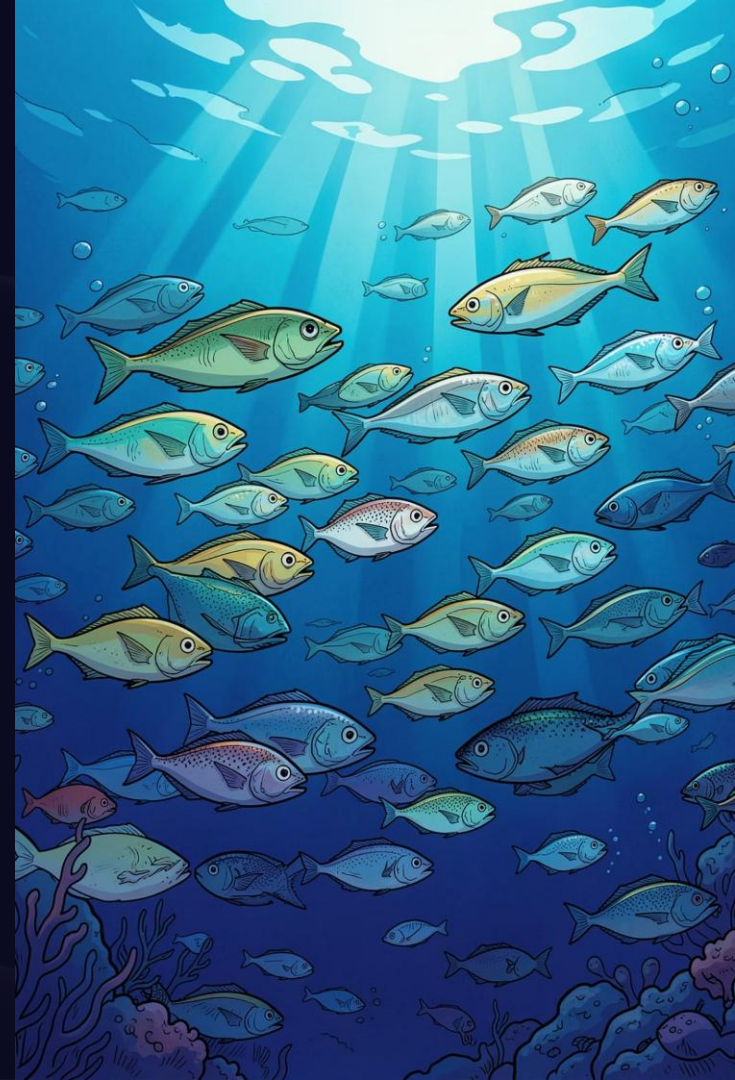
Utilizando **Babylon.js** — Simulación de comportamiento colectivo en tiempo real mediante el algoritmo Boids de Craig Reynolds

Universidad Autónoma Metropolitana

UEA: SEMINARIO DE TECNOLOGIAS DE LA INFORMACION II

- **Integrantes:**

- GONZALEZ RODRIGUEZ JOSE ALBERTO
- MARTINEZ BARRALES OSCAR
- MEJIA GARCIA OSWALDO



Objetivos del Proyecto

El propósito central de este proyecto fue desarrollar una simulación de 3D de un cardumen utilizando el algoritmo Boids y Babylon.js aplicando técnicas modernas de computación gráfica para mantener un comportamiento natural y un alto rendimiento en tiempo real.

Optimización de rendimiento

Diseñar una arquitectura capaz de simular cientos de agentes simultáneamente manteniendo una tasa de cuadros estable y una experiencia fluida e interactiva.

Shaders personalizados

Implementar shaders GLSL para animar las aletas y colas de los peces directamente en la GPU, liberando a la CPU de cálculos costosos y mejorando el rendimiento general.

Interactividad en tiempo real

Crear una interfaz que permitiera al usuario modificar los parámetros del algoritmo y observar sus efectos de manera inmediata, fomentando la exploración y el análisis.

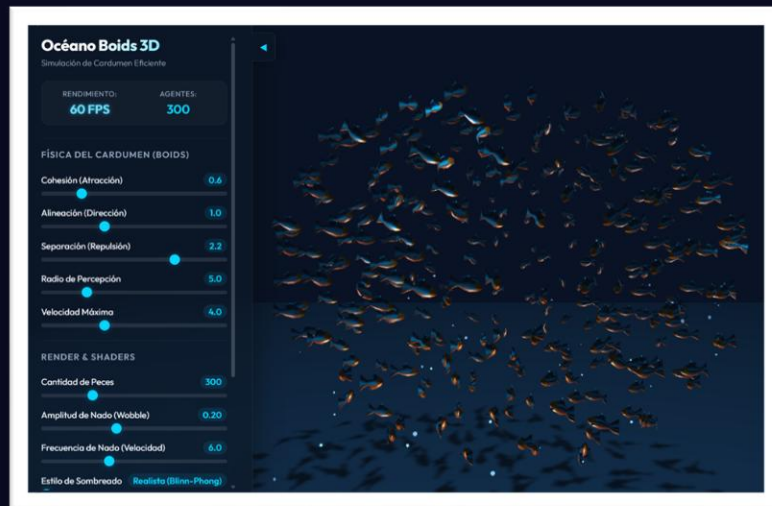
¿Qué Desarrollamos?

El proyecto consiste en una **simulación marina tridimensional** donde cada pez actúa como un agente autónomo e inteligente. A diferencia de animaciones predefinidas, cada agente evalúa su entorno local en cada cuadro de simulación y toma decisiones independientes basadas en la posición y velocidad de sus vecinos más cercanos.

Parámetros ajustables en tiempo real

El usuario puede modificar interactivamente los siguientes parámetros desde la interfaz gráfica:

- **Cohesión:** fuerza de atracción hacia el centro del grupo
- **Separación:** distancia mínima entre agentes
- **Alineación:** tendencia a seguir la dirección del grupo
- **Velocidad:** velocidad máxima de los peces
- **Radio de percepción:** distancia máxima de detección de vecinos
- **Cantidad de peces:** número de agentes activos
- **Tipo de sombreado:** Blinn-Phong o Cel Shading
- **Intensidad de cáusticas:** efecto de luz submarina

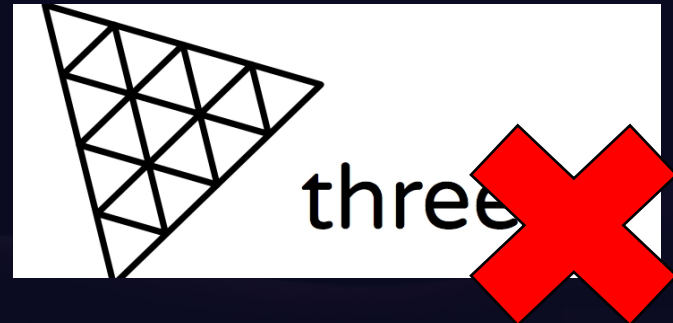


¿Por Qué Elegimos Babylon.js?

Antes de iniciar el desarrollo, se evaluaron dos motores gráficos para WebGL: **Three.js** y **Babylon.js**. Ambos cuentan con amplia documentación. Y después de un análisis de sus capacidades específicas para este proyecto, se llegó a la decisión de que Babylon.js ofrecía ventajas para una simulación de esta escala y complejidad.

Babylon.js fue la mejor opción porque ofrece:

- Thin Instances, ideales para renderizar cientos de peces utilizando una sola llamada de dibujo.
- Excelente soporte para Shaders personalizados, permitiendo animar los peces directamente en la GPU.
- Sistema de iluminación y sombras integrado.
- Mayor facilidad para desarrollar simulaciones interactivas en WebGL.



¿Por Qué un Cardumen de Peces?

Un cardumen de peces representa uno de los ejemplos más complejos visualmente para implementar el algoritmo Boids. En la naturaleza, miles de peces se mueven de manera coordinada sin un líder central, sin comunicación explícita y sin un plan global. Este fenómeno de **comportamiento emergente** es exactamente lo que se busca replicar computacionalmente.

Razones para elegir este escenario

- Comportamiento emergente
Las reglas locales simples generan patrones globales complejos y orgánicos, ideales para demostrar el poder del algoritmo.
- Interacción entre agentes
Cada pez interactúa únicamente con sus vecinos más cercanos, lo que permite evaluar la eficiencia de estructuras espaciales como el Spatial Hashing.
- Movimiento natural
El entorno acuático tridimensional permite explorar el movimiento en los tres ejes, añadiendo complejidad respecto a simulaciones 2D.
- Escenario visualmente atractivo
El ambiente submarino ofrece oportunidades únicas para aplicar cáusticas, partículas, iluminación volumétrica y shaders personalizados.



Funcionamiento del Algoritmo Boids

¿Qué es Boids?

El algoritmo Boids, propuesto por Craig Reynolds en 1986, permite simular el comportamiento colectivo de animales como peces, aves o insectos mediante reglas simples aplicadas a cada individuo.

En lugar de controlar todo el grupo, cada pez toma decisiones de manera independiente observando únicamente a los individuos que tiene cerca. La interacción de todos los peces genera un comportamiento emergente muy similar al observado en la naturaleza.

Las tres reglas.

1. Cohesión

El pez intenta mantenerse cerca del centro del grupo.

2. Alineación

Ajusta su dirección para nadar en la misma orientación que sus vecinos.

3. Separación

Evita colisiones manteniendo una distancia adecuada entre los peces.

Resultado.

La combinación de estas tres reglas permite que el cardumen se desplace de forma organizada, fluida y natural, sin necesidad de un líder o de un control central.

Tecnologías Implementadas

El proyecto integra un conjunto de tecnologías modernas que trabajan de forma coordinada para lograr una simulación computacionalmente eficiente. A continuación se describen los componentes principales del stack tecnológico:



Babylon.js — Motor Gráfico

Motor de renderizado 3D basado en WebGL que proporciona la infraestructura completa para la escena: cámara, iluminación, materiales, sistema de partículas y gestión del ciclo de renderizado.



Thin Instances — Renderizado Masivo

Técnica de renderizado que permite dibujar cientos o miles de copias del mismo modelo geométrico (los peces) con una única llamada a la GPU. Cada instancia tiene su propia transformación (posición, rotación, escala).



Spatial Hashing — Búsqueda de Vecinos

Estructura de datos espacial que divide el espacio 3D en celdas de una rejilla hash. En lugar de comparar cada pez con todos los demás, solo se comparan los peces dentro de celdas adyacentes.



Shaders GLSL — Animación en GPU

Los movimientos de las aletas y la cola de los peces se calculan directamente en el **Vertex Shader** mediante funciones sinusoidales parametrizadas por el tiempo.

Modelos de iluminación

- **Blinn-Phong:** Modelo de iluminación clásico que produce superficies realistas con specular highlights suaves, ideal para simular las escamas de los peces.
- **Cel Shading:** Modo de sombreado no fotorrealista (NPR) que genera un estilo visual tipo caricatura o ilustración, ofreciendo variedad estética al usuario.

Efectos visuales adicionales

- **Sistema de partículas:** Generación de burbujas ascendentes que añaden dinamismo y profundidad visual a la escena submarina.
- **Fondo procedural:** El lecho marino se genera mediante funciones de ruido matemático (Perlin/Simplex), creando un terreno único en cada ejecución sin necesidad de assets externos.

El problema y la propuesta

Idea principal: *Lograr un equilibrio entre realismo y eficiencia computacional.*

El problema.

La animación de personajes anatómicos es un reto en computación gráfica. Mientras que animar únicamente la superficie de un modelo es relativamente sencillo, representar de forma conjunta la piel, los músculos y el esqueleto suele provocar deformaciones poco realistas, intersecciones entre tejidos o un elevado costo computacional.



La propuesta.

El artículo presenta Neural Deformation Gradients (NDG), una técnica basada en redes neuronales que aprende cómo se deforman los diferentes tejidos del cuerpo humano. Gracias a este enfoque, es posible obtener animaciones de alta calidad con un rendimiento suficiente para ejecutarse en tiempo real.

¿Cómo funciona Neural Deformation Gradients?



El modelo representa el cuerpo humano mediante tres capas conectadas:

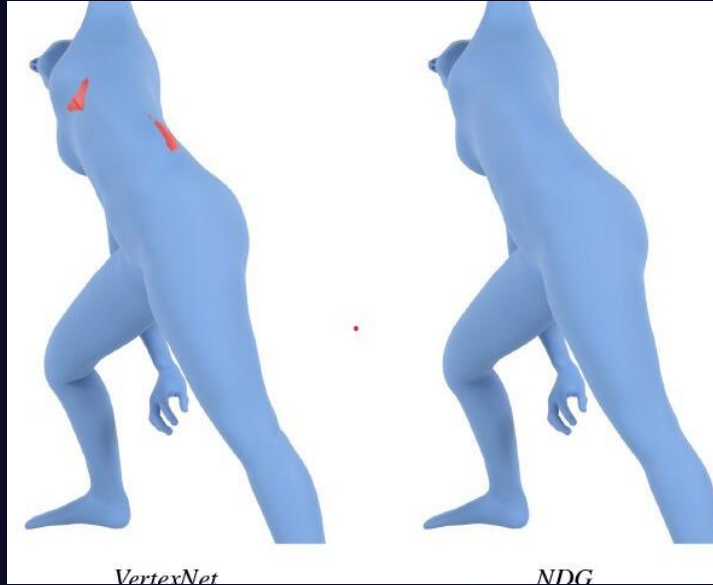
- Esqueleto, que define la estructura y el movimiento principal.
- Músculos, que transmiten y deforman el movimiento.
- Piel, que refleja el resultado final de la deformación.

En lugar de calcular directamente la posición de cada vértice, la red neuronal predice gradientes de deformación, es decir, aprende cómo se estira, comprime y rota cada parte del material durante el movimiento.

Además, el método incorpora una formulación matemática que evita deformaciones imposibles, garantizando que la geometría conserve su volumen y no presente inversiones o errores visuales.

Resultado: movimientos más naturales y físicamente consistentes.

Resultados y conclusión del artículo



Los experimentos muestran que el método propuesto ofrece mejoras importantes respecto a técnicas anteriores:

- Reduce más de un 50 % las intersecciones entre tejidos.
- Presenta un comportamiento más robusto que otros modelos recientes, como SKEL y HIT.
- Generaliza correctamente a modelos anatómicos distintos de los utilizados durante el entrenamiento.
- Mantiene un rendimiento cercano a 30 FPS, permitiendo su uso en aplicaciones de tiempo real.

Relación con nuestro proyecto.

Aunque el artículo se enfoca en la animación de cuerpos humanos y nuestro proyecto en la simulación de un cardumen de peces, ambos comparten un mismo objetivo: utilizar técnicas avanzadas de computación gráfica para obtener animaciones realistas manteniendo un alto rendimiento en tiempo real.

Conclusiones

- Se logró implementar una simulación de un cardumen utilizando el algoritmo Boids, obteniendo un movimiento natural mediante reglas simples de comportamiento.
- La elección de Babylon.js facilitó el desarrollo de la escena y permitió aprovechar herramientas de optimización como Thin Instances y Shaders, manteniendo un buen rendimiento.
- El proyecto demostró la importancia de combinar algoritmos de simulación con técnicas de renderizado para obtener resultados visualmente atractivos y eficientes.
- Finalmente, el artículo analizado mostró cómo las investigaciones actuales continúan mejorando la calidad de las animaciones mediante inteligencia artificial, reforzando la relevancia de la computación gráfica en áreas como los videojuegos, la medicina y la visualización científica.





***GRACIAS POR SU
ATENCIÓN***